

Understanding RAG — AI Jargon Simplified #1

S

Harisathvika

8 min read

.

21 hours ago

5

We all listen to these complex-sounding terms like RAG, embeddings, vector databases, fine tuning, tokens etc. As a CS student learning these concepts myself, I realized that many explanations online feel overly technical and difficult for beginners to follow

So I started AI Jargon Simplified — a series of blogs where I break down AI concepts into simple, beginner-friendly explanations using analogies, visuals, and practical examples.

This series is not written from the perspective of an expert researcher, but from a learner trying to truly understand how AI systems work — one concept at a time.

Contents

- #1 What is RAG?
- #2 How Does RAG Work?
- #3 What are Tokens?
- #4 Vector Embeddings
- #5 What is a Vector Database?
- #6 What is Semantic Search?
- #7 Cosine Similarity and Similarity Scores
- #8 Retrieval — Augmentation — Generation
- #9 Intro to LangChain
- #10 Resources to Learn More and Build Stuff

#1 What is RAG?

LLMs are powerful, but they have a major limitation. they only know what they were trained on. Imagine asking an AI model

trained only on data until 2023 “What is BTS’s new Album name”.

No! it won’t answer Arirang. Because it was never trained on that data. The model may give outdated info or even hallucinate an incorrect answer. Large Language Models are trained mostly on public internet data like websites, articles, books, forums, public code repositories etc. The most valuable business data is private.

Now imagine a company wants an AI assistant that can answer questions about its internal policies or something. A normal LLM can’t answer questions like “What is our company’s leave policy?” since that info was never part of its training data.

Instead of retraining the entire model on confidential documents, RAG retrieves the relevant internal documents in real-time and provides them to the LLM.

So, simply, RAG (Retrieval-Augmented-Generation) is when you change the output of an LLM by providing the model more context

along side a user's input. This is cost effective and enables model to give up-to-date and accurate answers.

#2 How does all of this work?

Press enter or click to view image in full size

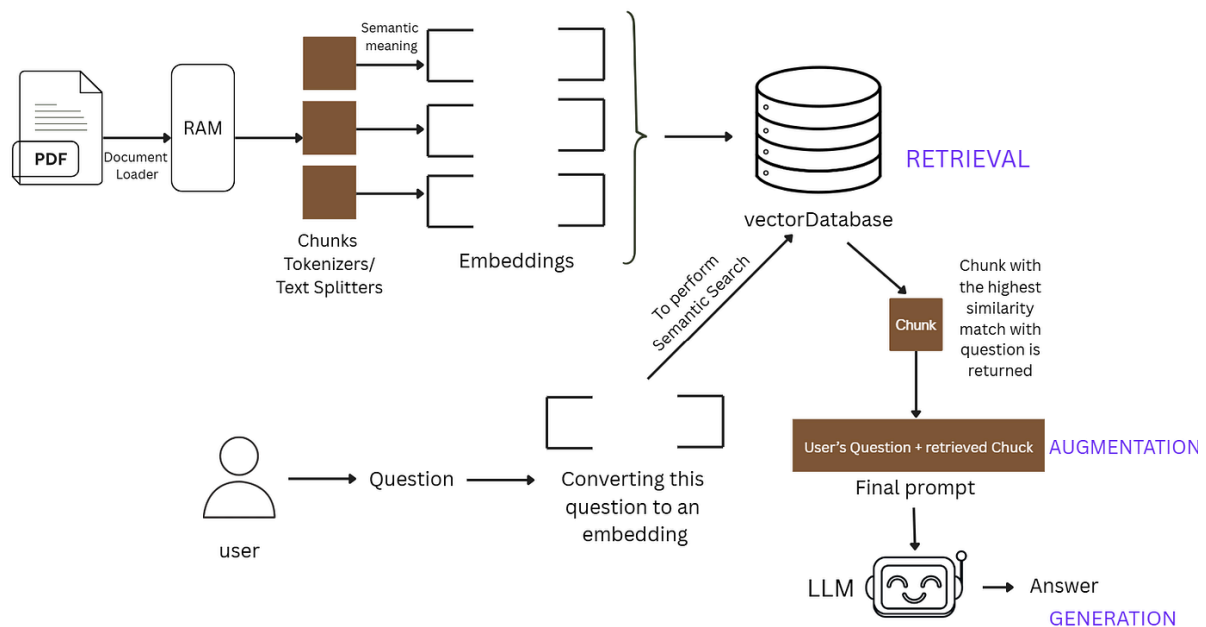


Fig. RAG WORKING

Let us only look at the big picture for now and decode the jargon later,

- documents are stored
- converted into searchable meaning representations

- relevant information is retrieved
- AI uses it to answer

#3. What are Tokens?

A token is a small unit of text that an AI model processes. A token can be, a word, part of a word, a character or a punctuation. AI models do not read text exactly like humans do. They read and process tokens.

Tokenizers?

A tokenizer is the system that converts normal text into tokens. It acts like a translator between human language and machine-readable input. Without tokenizers, LLMs cannot process text.

In RAG pipelines, documents are split into chunks using tokenizers. This helps in fitting information into context windows(The context window of an LLM is the amount of text, in tokens, that the model

can consider or “remember” at any one time), improve retrieval quality, avoid oversized prompts

#4 Vector Embeddings

Computers do not understand language like humans do. The AI needs a way to mathematically represent the meaning to preserve context. That’s where embeddings come in.

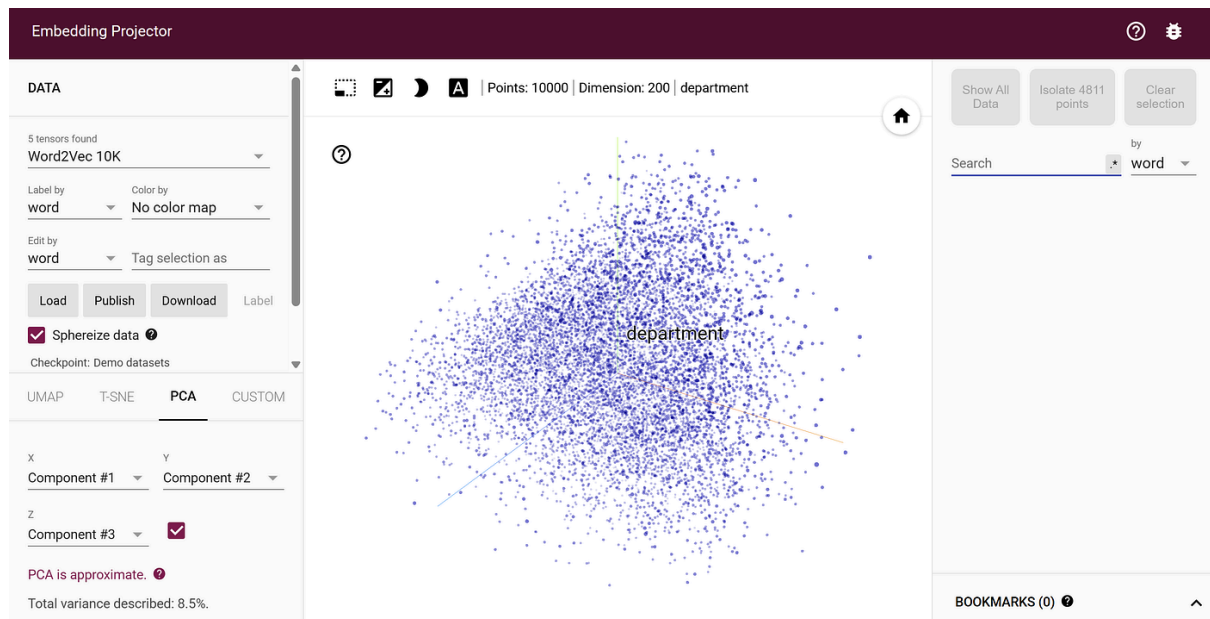
Imagine every word is placed on a giant map and words with similar meanings are placed closer together.

Example:

- “cat” and “kitten” → close
- “apple” and “os” → close
- “pizza” and “galaxy” → far apart

Embeddings help AI create this “meaning map.”

Press enter or click to view image in full size

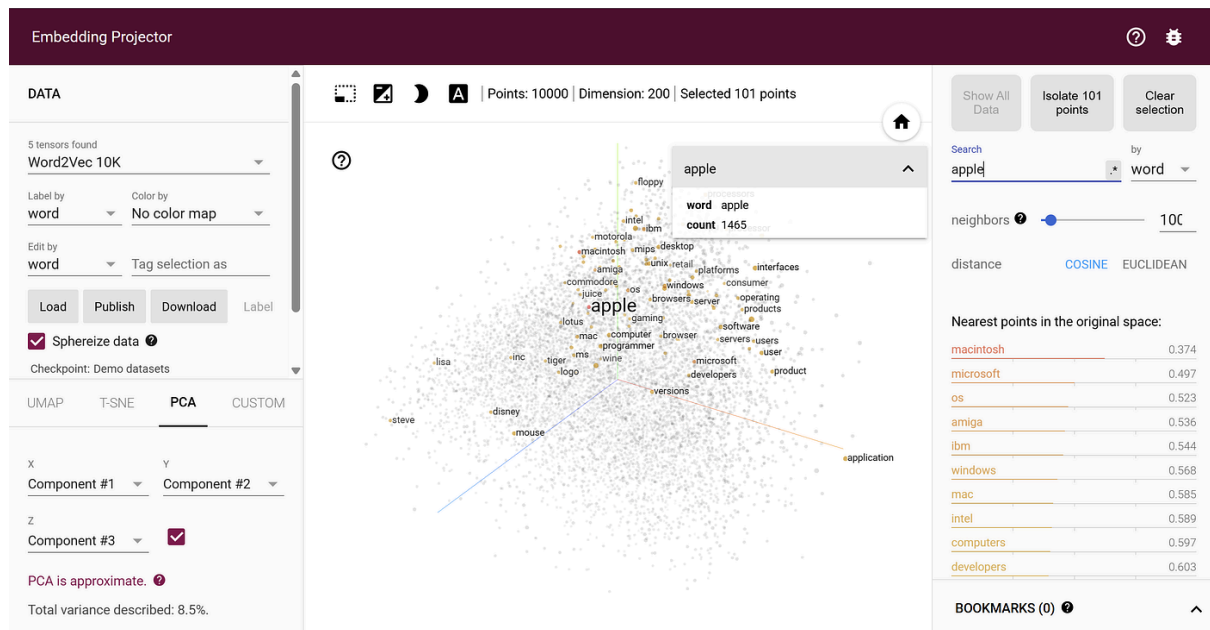


One of the coolest ways to actually *understand* embeddings is by visualizing them. Try it yourself here

<https://projector.tensorflow.org/>

In this example below, I searched for the word “apple” inside the TensorFlow Embedding Projector. What’s interesting is that the system didn’t just return random words, it returned words that are semantically related to “apple” in the context of technology and computing.

Press enter or click to view image in full size



Vector embedding of word “Cat” may look like.

Press enter or click to view image in full size

Text 1 Embedding

3072D vector

First 10 dimensions:

[-0.0189, 0.0082, 0.0269, -0.0684, 0.0008, 0.0330, -0.0286, 0.0133, 0.0122, 0.0036 ...]

These embeddings may vary from model to model. These numbers are not universal.

Some of the embedding techniques are Word2vec, BERT etc.

Now that we can convert text into embeddings, another question appears. Where do we store all these embeddings? How does the AI quickly search through millions of them to find the most relevant information? That's where vector databases come into picture

#5 What is a Vector Database?

A vector database is a specialized database designed to store and search embeddings efficiently. According to IBM, A vector database stores, manages and indexes high-dimensional vector data(Let's come to this dimensionality a bit later). In a vector database, data points are stored as arrays of numbers called "vectors," which can be compared and clustered based on similarity.

How a Vector Database Works

1. Data Ingestion

First, the system takes data such as, documents, sentences, images, user queries and prepares it for processing.

2. Creating Embeddings

The data is then converted into embeddings.

3. Storing and Organizing Vectors

These embeddings are stored inside a vector database. The database organizes them in a way that makes searching extremely fast, even when dealing with millions of vectors.

4. Similarity Search

When a user asks a question, the query is also converted into an embedding. The vector database then compares this query embedding with stored embeddings to find the most semantically similar matches. Instead of searching for exact keywords, it searches for similar meanings.

5. Retrieval

Finally, the most relevant results are retrieved and sent to the AI system. This is what allows applications like AI chatbots, recommendation systems, semantic search engines and RAG pipelines to provide intelligent and context-aware responses.

#6 What is Semantic Search?

Semantic search focuses on understanding the *meaning* and *context* behind a query rather than just matching exact keywords.

This is possible because both documents and user queries are converted into embeddings and placed in a vector space. If two embeddings are close together in that space, the AI considers them semantically similar.

This is why modern AI systems can understand that “car” and “automobile”, “movie” and “film”, “doctor” and “physician” may refer to similar concepts even though the words are different.

let us understand semantic search in-deapth.

Imagine how recommendation systems on platforms like Netflix work.

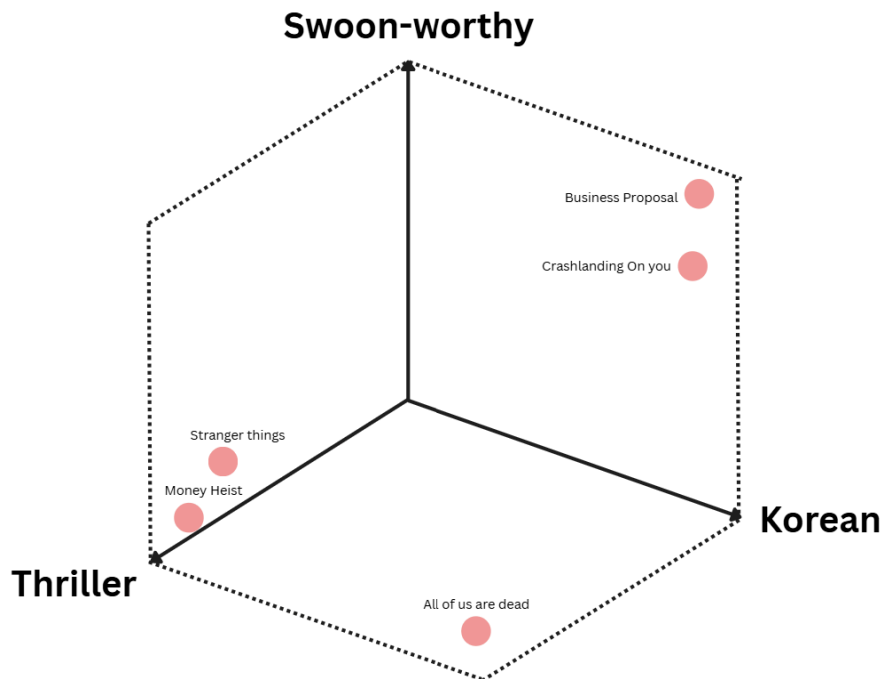
Instead of simply grouping content by genre, AI systems can place movies and shows inside a multi-dimensional “meaning space” based on their characteristics.

For example, imagine properties of a content as dimensions, like:

- Swoon-Worthy Romance
- Thriller
- Korean
- Horror
- Quirky/Comedy
- Sports Event

Now different shows can be positioned based on how strongly they match these characteristics.

Press enter or click to view image in full size



Netflix recommender system example

In this simplified visualization, shows are plotted based on characteristics like: thriller intensity, swoon-worthiness, Korean content

Shows with similar themes naturally appear closer together in the vector space. Real AI embeddings may contain hundreds or thousands of dimensions instead of just 3, but the core idea remains the same.

Similar meanings appear closer together. Imagine trying to manually organize millions of shows based on thousands of

characteristics at the same time. It would be nearly impossible. This is why AI systems use vector embeddings and similarity search.

Instead of relying only on exact genres or keywords, the AI compares patterns, themes, and relationships between shows to recommend content that feels similar to the user.

How does the AI determine whether two embeddings are similar?

#7 Cosine Similarity and Similarity Scores

Cosine similarity is a mathematical technique used to measure how similar two vectors are by comparing the angle between them in high-dimensional space.

Instead of focusing on the exact numbers inside embeddings, cosine similarity focuses on the *direction* of the vectors.

Imagine two arrows:

- If both arrows point in almost the same direction → they are considered very similar.
- If the arrows point in completely different directions → they are considered less similar.

This comparison produces something called a similarity score.

For example:

Crash Landing on You and Business Proposal may produce a high similarity score because both are romance-heavy Korean dramas.

Business Proposal and Money Heist may produce a lower similarity score because their themes are very different.

Press enter or click to view image in full size

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}},$$

#8 Retrieval — Augmentation — Generation

Retrieval

Now let's connect all the pieces together and understand what actually happens inside a RAG system when a user asks a question.

Step 1: The User Query is Converted into an Embedding, semantic meaning of question is captured

Step 2: Similarity Search Happens: The vector database compares the query embedding with the embeddings of stored document chunks. Using cosine similarity, the system calculates which chunks are semantically closest to the user's query.

Step 3: Relevant Chunks are Retrieved

Augmentation

The retrieved information is then added alongside the user's original query. This process is called Augmentation

Generation

Once the relevant document chunks are retrieved and added to the prompt, the final step begins.

At this stage, the Large Language Model receives, the user's original query and the retrieved contextual information and generates a response based on both.

Instead of answering purely from memory, the model now has access to relevant external knowledge while generating the response. This is what makes RAG systems powerful.

#9 Where Does Langchain Come Into the Picture?

we have understood the major building blocks of a RAG system:

- tokenization
- embeddings
- vector databases
- semantic search

- retrieval
- augmentation
- generation

But in real-world applications, all these components need to work together as a complete pipeline. This is where frameworks like Langchain come into the picture.

Langchain helps developers connect different parts of an AI application, such as **document loaders, embedding models, vector databases, retrievers** and **LLMs**

We'll explore Langchain and how it helps build real-world AI pipelines in a future blog of AI jargon simplified series.